ROBUST PROTECTION PLUG-IN FOR CLOUD STORAGE

A TECHNICAL MINI PROJECT REPORT

Submitted by

KARTHIKEYAN N (8208E21BSR028)

MOHAMED HASHIF H (8208E21BSR034)

SANTHOSH M (8208E21BSR050)

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND BUSINESS SYSTEMS

E.G.S. PILLA Y ENGINEERING COLLEGE, NAGAPATTINAM.

An Autonomous Institution; Affiliated to

ANNA UNIVERSITY :: CHENNAI 600 025

DEC 2024

E.G.S. PILLAY ENGINEERING COLLEGE, NAGAPATTINAM.

An Autonomous Institution; Affiliated to

ANNA UNIVERSITY: CHENNAI 600 025.

BONAFIDE CERTIFICATE

Certified that this Project report "**ROBUST PROTECTION PLUG-IN FOR SENSITIVE CLOUD STORRAGE DATA"** is the bonafide work of **KARTHIKEYAN N-E21BSR028, MOHAMED HASHIF H-E21BSR034, SANTHOSH M-E21BSR050"** who carried out the Project Work under my supervision.

SIGNATURE

Dr. P. ANANDRAJ ,M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Department of Computer Science and Business Systems.

E.G.S Pillay Engineering College (Autonomous)

Nagapattinam.

SIGNATURE

Dr. P. ANANDRAJ ,M.E., Ph.D.,

SUPERVISOR

ASSISTANT PROFESSOR

Department of Computer Science and Business Systems.

E.G.S Pillay Engineering College(Autonomous)

Nagapattinam.

Submitted for the Autonomous End Semester Examination Technical Mini Project viva - voce held on _____.

IN TERNAL EXAMINER **EXTERNAL EXAMINE**

ABSTRACT

This project presents a browser extension designed to enhance the security of cloud storage through automated encryption and decryption of files. Leveraging the **AES-256 encryption algorithm**, the extension ensures that user data remains private and secure, even in the event of cloud breaches, browser exploits, or unauthorized access to cloud accounts. The project emphasizes simplicity, supporting batch file processing, checksum generation, and seamless integration with popular cloud platforms like **Google Drive** and **OneDrive**. The extension provides a **user-friendly interface** with drag-and-drop functionality and password management features. It allows users to set and save a **master password** for encrypting and decrypting files while offering tips to create strong yet memorable passwords. By working as a browser extension, it acts as a universal solution compatible with various cloud platforms, utilizing their web-based upload and download capabilities. this extension prioritizes **data privacy and integrity**, ensuring that encrypted files cannot be accessed without the master password. Its batch processing feature improves efficiency for handling multiple files simultaneously. The use of checksum validation further ensures the integrity of data during encryption and decryption processes. This project is ideal for addressing the growing concerns surrounding **cloud data security**, offering a minimal, practical, and effective solution. Designed with non-technical users in mind, it balances ease of use with robust security features, making it an innovative approach to securing cloud storage for individuals and small organizations alike

Keywords: AES-256 Encryption, Cloud Storage Security, Browser Extension, Data Privacy, Checksum Validation, Drive Integration, File Encryption, Automatic Decryption, Cloud Data security, File Integrity.

TABLE OF CONTENT

LIST OF FIGURES

CHAPTER I

INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

This project is an innovative browser extension designed to secure user data stored in cloud platforms. This project addresses the growing demand for robust data security as individuals and organizations increasingly rely on cloud services like Google Drive and OneDrive for file storage. Leveraging **AES-256 encryption**, a highly secure and industry-standard algorithm, Cloud Crypt ensures that sensitive information remains confidential, even in worst-case scenarios such as cloud breaches or unauthorized access to accounts.

The extension automatically encrypts files before uploading them to cloud storage and decrypts them upon retrieval. This seamless process eliminates the risk of unencrypted data exposure. Designed for ease of use, the extension supports drag-and-drop functionality, batch file processing, and password management. The inclusion of checksum validation further enhances reliability by ensuring file integrity throughout the encryption and decryption processes.

With its simple yet effective interface, Cloud Crypt empowers users to take control of their data security. Whether it's personal photos, business documents, or other multimedia files, this tool provides peace of mind by ensuring that files are accessible only to authorized users. By acting as a common solution compatible with multiple cloud platforms, the project aims to make advanced data security accessible to everyone, regardless of technical expertise.

The key objectives of this project include:

1. **Enhancing Cloud Data Security**
   The primary objective of project is to safeguard files stored in cloud platforms by utilizing AES-256 encryption. This ensures that even if cloud servers are breached or login credentials are compromised, encrypted files remain inaccessible without the user’s master password.

2. **Streamlined User Experience**
   The project prioritizes simplicity and accessibility. With drag-and-drop functionality, intuitive file selection options, and batch processing capabilities, Cloud Crypt ensures that non-technical users can easily secure their data without navigating complex procedures.

3. **Universal Compatibility**
   this plug-in is designed as a browser extension to act as a universal solution compatible with various cloud platforms. By integrating with web-based upload and download processes, it eliminates the need for platform-specific applications, making it versatile and scalable.

4. **Password and Data Integrity Management**
   The extension enforces strong password policies (8–12 characters) while offering tips for creating secure yet memorable passwords. It also incorporates checksum validation to verify data integrity, ensuring that files are not corrupted during encryption or transfer.

5. **Batch Processing for Efficiency**
   To address the needs of users dealing with large datasets, the extension supports batch processing, allowing multiple files to be encrypted, uploaded, and decrypted simultaneously. This feature enhances productivity without compromising security.

6. **Future Scalability**
   The project is designed with scalability in mind, allowing for future integration with additional cloud platforms and expanded functionality, such as real-time synchronization of encrypted data or mobile app extensions.

7. **Promoting Awareness of Data Privacy**
   Cloud Crypt aims to promote awareness about the importance of data privacy in the digital age. By making encryption accessible, it empowers users to proactively secure their sensitive information.

1.2 SCOPE OF THE PROJECT

The browser extension is a robust, **security-focused tool** aimed at safeguarding user data stored on cloud platforms. Its design ensures maximum compatibility with various cloud storage services while providing an intuitive and user-friendly experience. By focusing on advanced encryption standards, automated processes, and seamless cloud integration, this project addresses the increasing demand for secure data storage in both personal and professional contexts. The scope of the project extends from addressing immediate user needs to creating a scalable solution capable of evolving with future technological advancements.

The scope of the Cloud project spans from addressing immediate challenges in cloud data security to providing a scalable and future-ready solution. Its focus on user-friendly design, robust encryption, and **platform compatibility** ensures that it is both practical and impactful, catering to diverse user needs and establishing a strong foundation for continued development.

TECHNICAL SCOPE :

Automation and Usability:
The extension focuses on minimizing manual effort through automation. Features like drag-and-drop functionality and real-time encryption ensure that users can easily and securely handle their files.

Scalability and Future Enhancements:
The project is designed to be scalable, with future updates potentially including real-time encrypted file synchronization, mobile application versions, and enhanced user analytics for better password management and usage insights.

Cross-Platform Compatibility:
Designed as a browser extension, Cloud Crypt operates independently of operating systems, making it accessible on various devices and platforms. It supports all major browsers, including Chrome, Edge, and Firefox, and ensures universal functionality.

POTENTIAL SCOPE :

Data Privacy and Security:
By ensuring files are encrypted before uploading to the cloud, the extension protects sensitive data against potential breaches, unauthorized access, and browser exploits.

Target Users:
The tool is designed for a broad audience, including students, small businesses, and professionals seeking an easy-to-use, secure solution for cloud storage.

Industry Relevance:
With growing concerns about data privacy, Cloud Crypt aligns with global trends and regulatory requirements, such as GDPR, promoting responsible and secure data management practices

FUNCTIONAL SCOPE :

File Encryption and Decryption:
The extension automatically encrypts files with AES-256 encryption, a military-grade standard known for its robustness, before uploading them to cloud platforms. It decrypts these files upon download, ensuring a seamless experience for users while maintaining maximum security. This functionality covers a wide range of file types, including multimedia files and documents.

Password Management:
The extension enforces strong password policies, requiring users to create master passwords that are 8–12 characters long. It also offers tips for crafting secure yet memorable passwords. Passwords are securely saved for future decryption, along with checksum validation to ensure data integrity and prevent corruption during encryption or transfer.

Cloud Platform Integration:
Cloud Crypt integrates with major cloud storage platforms such as Google Drive and OneDrive through their APIs. This ensures a seamless process for uploading and downloading encrypted files. The browser extension leverages the universal nature of web-based uploads and downloads, making it a common solution across multiple cloud platforms. Future versions can expand compatibility to include other popular cloud services like Dropbox and iCloud.

1.3 DATA SECURITY IN CLOUD COMPUTING

Data security in cloud computing focuses on protecting sensitive information stored and processed in cloud environments from unauthorized access, breaches, and potential loss. As organizations and individuals increasingly rely on cloud storage, ensuring data confidentiality, integrity, and availability becomes critical. Below are four key subtopics that highlight the dimensions of data security in cloud computing.

DATA ENCRYPTION **:**

Encryption is the cornerstone of data security in cloud computing. By converting sensitive information into unreadable ciphertext, encryption ensures that even if data is intercepted, it cannot be accessed without the correct decryption key.

- **At-Rest Encryption**: Protects stored data by encrypting it on cloud servers. AES-256 is a widely used standard for this purpose.
- **In-Transit Encryption**: Secures data during transfer between devices and cloud platforms using protocols like TLS (Transport Layer Security).
- **Challenges**: Managing encryption keys securely is essential to prevent unauthorized access.

ACCESS CONTROL AND AUTHENTICATION:

Access control mechanisms define who can access specific data and under what conditions, ensuring that only authorized users have access to sensitive information.

- **Multi-Factor Authentication (MFA)**: Adds an extra layer of security by requiring multiple verification methods, such as passwords and OTPs.
- **Role-Based Access Control (RBAC)**: Assigns permissions based on user roles, limiting data exposure.
- **Challenges**: Balancing security and usability, especially for non-technical users.

DATA INTEGRITY AND BACKUP :

Data integrity ensures that the stored information remains unchanged and uncorrupted. Regular backups protect against accidental deletions, ransomware attacks, or server failures.

- **Checksum Validation**: Verifies the integrity of data by comparing pre-upload and post-download checksums.
- **Redundancy**: Cloud providers often replicate data across multiple locations to ensure availability.
- **Challenges**: Ensuring timely backups and testing restoration procedures to avoid downtime.

COMPLIANCE AND LEGAL CONSIDERATIONS:

Cloud data security must comply with global regulations and standards that govern data protection and privacy. These ensure ethical and legal handling of user data.

- **Regulatory Frameworks**: Examples include GDPR (General Data Protection Regulation) in the EU and HIPAA (Health Insurance Portability and Accountability Act) in the US.
- **Cloud Provider Policies**: Providers must adhere to legal standards and maintain transparency about how they handle data.
- **Challenges**: Organizations need to verify that their chosen cloud provider complies with relevant regulations.

Data security in cloud computing is multi-faceted, requiring robust encryption, controlled access, data integrity checks, and adherence to legal standards. By focusing on these subtopics, users and organizations can ensure their data remains secure while leveraging the flexibility and scalability of cloud services.

1.4 BROWSER EXTENSION FOR SECURING CLOUD STORAGE FILES

A browser extension for securing cloud storage files offers a straightforward solution to protect sensitive data in cloud environments. By integrating encryption and decryption mechanisms directly into the browser, this extension enhances security without requiring users to interact with complex tools or interfaces. Below is a detailed breakdown of the key elements involved in building such a browser extension.

1. Encryption Mechanism: AES-256 for Robust Security

The core functionality of the extension involves AES-256 encryption, a widely recognized and secure encryption algorithm. AES-256 ensures that files uploaded to cloud storage are encrypted locally before they are transferred. The extension encrypts files automatically when users select them for upload, ensuring that even if the cloud service is compromised, the data remains unreadable without the encryption key. AES-256 offers a high level of security, making it one of the most reliable encryption methods available.

Why AES-256?:

AES-256 offers a good balance between speed and security, making it ideal for encrypting large files without significant performance penalties. Its strength in encryption protects against brute-force attacks and unauthorized access, ensuring that sensitive user data remains confidential.

Access Control and User Consent

2. Seamless Integration with Cloud Storage Platforms

The browser extension works directly within the browser environment, allowing users to interact with cloud storage platforms such as Google Drive, OneDrive, or Dropbox. The extension integrates with the cloud's **web upload/download APIs** to automate the encryption and decryption processes.

- **Automatic Encryption**:
  As users select files for upload, the extension automatically encrypts the files before sending them to the cloud storage platform, without requiring any additional steps from the user.

- **Automatic Decryption**:
  When files are downloaded, the extension decrypts them seamlessly, ensuring that users only need to access the files in their decrypted form. This functionality ensures that users can interact with their files just as they would with unencrypted files, but with the added assurance of robust encryption.

3. Data Integrity and Backup with Checksum Validation

To ensure that files are not corrupted during encryption or transmission, the extension uses **checksum validation**. This process calculates a checksum (a hash value) for each file before encryption, and verifies the integrity of the file after it is uploaded and later downloaded.

- **File Integrity**:
  Checksum validation ensures that no data is lost or altered during the encryption, upload, or download process. If a file is corrupted or tampered with, the extension can alert the user, preventing the use of compromised files.

- **Backup Mechanism**:
  In addition to encryption, users can choose to back up their encrypted files to other locations, providing added security in case their primary cloud storage account is compromised or inaccessible.

4. User-Friendliness and Accessibility

The extension is designed with non-technical users in mind, ensuring that securing cloud storage files is as simple as possible. It offers **drag-and-drop functionality** for easy file uploads, intuitive password management features, and a straightforward interface.

- **Drag-and-Drop**:
  Users can simply drag files into the browser extension interface for immediate encryption, followed by upload to their chosen cloud storage service. This intuitive approach removes barriers to secure cloud file management.

- **Password Management**:
  The extension prompts users to create a **master password** (typically 8-12 characters) to secure their encrypted files. Password guidelines and tips help users choose a strong yet memorable password. Once set, the password is saved securely for future decryption processes, eliminating the need to re-enter it every time.

CHAPTER 2

LITERATURE SURVEY

1. Title: "A Survey on Data Security in Cloud Computing"

Author(s): S. Subashini, V. Kavitha
Year: 2011

Problem:
Cloud computing services are highly prone to security threats, such as data breaches, unauthorized access, and data loss. There is a growing concern about the confidentiality, integrity, and availability of data stored on third-party cloud servers.

Solution:
The paper proposes a comprehensive survey on various data security measures for cloud computing, including encryption techniques, access control, and data integrity mechanisms. It specifically highlights **AES encryption** as a viable solution for securing data in transit and at rest. The study also emphasizes the importance of a multi-layered security approach combining encryption, authentication, and monitoring.

2. Title: "Cloud Storage Security: A Survey on Key Management and Cryptographic Protocols"

Author(s): L. Zhang, S. Wang, J. Zhang, Q. Sun
Year: 2014

Problem:
Data security in cloud storage is a major concern, particularly regarding key management and the encryption techniques used to protect sensitive files. Weak key management practices and poor encryption choices can lead to unauthorized data access and security breaches.

Solution:
This paper reviews key management systems (KMS) and cryptographic protocols specifically designed for cloud storage security. The authors highlight **AES-256 encryption** for its strong security properties and discuss various key management protocols to handle encryption keys effectively. The paper suggests using public-key cryptography alongside symmetric encryption for better performance and security.

3. Title: "Secure Data Storage and Privacy Protection in Cloud Computing"

Author(s): M. A. A. Azees, M. R. S. Anwar, N. V. P. D. Krishna
Year: 2015

Problem:
With the increased adoption of cloud computing, securing sensitive data stored on cloud platforms becomes essential. The challenge lies in ensuring **data confidentiality**, preventing unauthorized access, and providing **secure file retrieval**.

Solution:
The paper suggests a hybrid approach combining symmetric encryption (such as **AES-256**) for data encryption and **secure hash algorithms** for ensuring data integrity. It also introduces a novel approach for combining both **client-side encryption** and **server-side encryption** to strengthen data security, ensuring that sensitive data is encrypted before being uploaded to the cloud and only decrypted on the client-side.

4. Title: "Client-Side Encryption for Cloud Data Security: Challenges and Solutions"

Author(s): C. Wang, Q. Wang, K. Ren, W. Lou
Year: 2012

Problem:
Data confidentiality in cloud storage is compromised when users rely solely on service providers for encryption and key management. This creates vulnerabilities, as cloud providers may have unauthorized access to user data or could be subject to breaches.

Solution:
The paper emphasizes client-side encryption as a solution, where data is encrypted before being uploaded to the cloud. It discusses implementing strong encryption algorithms, including AES-256, and highlights the need for efficient key management solutions. Additionally, the study identifies challenges such as usability issues and performance overhead and proposes optimized techniques for secure and scalable encryption, including batch encryption for multiple files.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Most current cloud storage platforms, such as Google Drive, OneDrive, and Dropbox, provide basic security measures like **server-side encryption** (data is encrypted by the cloud provider after upload). While these measures protect data to some extent, they pose several limitations that compromise user privacy and data security.

LIMITATIONS OF THE EXISTING SYSTEM :

1. **Reliance on Service Providers**:
    - Users depend on cloud service providers for encryption and key management.
    - Providers have access to encryption keys, making data vulnerable to breaches or insider threats.
2. **Lack of Client-Side Encryption**:
    - Most platforms do not offer client-side encryption as a standard feature.
    - Users cannot encrypt their files locally before uploading them to the cloud.
3. **Data Breaches**:
    - If the cloud provider's systems are compromised, encrypted data can be decrypted using stolen keys.
    - High-profile breaches (e.g., ransomware attacks) demonstrate the risks of relying solely on provider-managed encryption.
4. **Limited Control Over Password Management**:
    - Users are not given control over encryption passwords or algorithms.
    - Weak or default passwords often increase the risk of unauthorized access.

5. **Integration Complexity for Custom Solutions**:

    - Users seeking higher security through third-party encryption tools face integration challenges with existing cloud platforms.
    - These tools often lack seamless compatibility and user-friendliness.

GAPS IN USER-CENTRIC SECURITY :

- **No Granular File Protection**: Encryption is typically applied at the account or folder level, limiting file-specific security.
- **Insecure File Sharing**: Shared links for files often bypass encryption, exposing data to unauthorized users.
- **No Real-Time Decryption**: Existing systems do not support real-time file decryption during download, requiring users to manually decrypt files after retrieval.

CHALLENGES ADDRESSED BY A NEW SYSTEM :

To overcome these shortcomings, a new system focused on **client-side encryption** with seamless **browser extension integration** is proposed. This system will enable users to encrypt files locally using **AES-256 encryption** before uploading them to the cloud, ensuring that even if the cloud or browser is compromised, data remains secure. Additionally, automated decryption upon download will enhance usability without sacrificing security.

This analysis highlights the need for a more secure, user-controlled, and accessible solution to address the vulnerabilities in existing cloud storage systems.

DISADVANTAGES :

1. **Lack of Full User Control**
    - Users rely on cloud providers for encryption and key management, reducing their control over their data.
    - Encryption keys are often stored with the provider, making data vulnerable to insider threats or misuse.
2. **Risk of Data Breaches**
    - Cloud storage systems are frequent targets of cyberattacks.
    - If the provider's servers are compromised, attackers can access encrypted data using stolen keys.
3. **No Default Client-Side Encryption**
    - Most existing systems lack client-side encryption, meaning data is uploaded in plaintext before being encrypted on the server.
    - This exposes sensitive information during transmission and increases the risk of interception.
4. **Limited Data Confidentiality**
    - Service providers have access to unencrypted data or decryption keys, allowing them to potentially view or misuse user data.
    - Government agencies can also demand access to data under legal frameworks.
5. **Vulnerability in File Sharing**
    - Sharing links or granting access to files often bypasses encryption, exposing data to unauthorized users.
    - Shared files may not maintain their encrypted state, increasing security risks.
6. **Inconsistent Encryption Standards**
    - Not all providers use robust encryption algorithms; some rely on outdated or less secure methods.
    - There’s a lack of transparency regarding encryption policies.
7. **Performance Bottlenecks**
    - Server-side encryption can lead to slower upload and download speeds, especially for large files.
    - Decryption on the server end adds latency, reducing efficiency.

8. **Integration Challenges**
    - Adding third-party encryption solutions to existing systems is often complicated and requires technical expertise.
    - Such integrations may disrupt the user experience or introduce compatibility issues.
9. **No Personalized Security Features**
    - Current systems lack features like user-defined encryption passwords or custom encryption algorithms.
    - Users cannot customize security settings to meet specific requirements.
10. **Data Integrity Risks**
    - Providers focus on encryption but often neglect data integrity measures, leading to potential corruption during uploads or downloads.

3.2 PROPOSED SYSTEM

The proposed system introduces a **browser extension** designed to secure files before they are uploaded to cloud storage. This extension, leveraging **AES-256 encryption**, ensures that data is encrypted locally on the user's device, providing full control over data security. Users can select files, encrypt them with a master password, and upload them directly to various cloud platforms. The extension also facilitates **batch processing**, allowing multiple files to be encrypted and uploaded efficiently.

Additionally, the system includes a **password and checksum management feature**, ensuring that encrypted files can only be decrypted with the correct password. Decryption is automated during file downloads, streamlining the process for users. The **drag-and-drop interface** ensures a user-friendly experience, making advanced security accessible to non-technical users.

This system overcomes the shortcomings of existing cloud security by eliminating dependency on providers for encryption, ensuring data remains secure even in the event of breaches, browser exploits, or unauthorized account access. By providing seamless integration with web-based cloud services and robust encryption, the proposed system ensures privacy, security, and ease of use.

ADVANTAGES:

1. **Enhanced Data Security**
    - Utilizes **AES-256 encryption**, ensuring robust protection for sensitive files.
    - Data remains secure even in the event of cloud breaches, browser exploits, or unauthorized access.
2. **User-Controlled Encryption**
    - Users encrypt and decrypt files locally, maintaining complete control over their data.
    - No reliance on cloud providers for key management or encryption processes.
3. **Cross-Platform Compatibility**
    - Functions as a **browser extension**, making it compatible with multiple cloud storage platforms like Google Drive, OneDrive, and Dropbox.
    - Avoids platform-specific limitations, providing a universal solution.
4. **Simplified User Experience**
    - Features a **drag-and-drop interface** and support for batch processing, ensuring ease of use for both technical and non-technical users.
    - Automatic decryption during downloads enhances convenience.
5. **Password Protection**
    - Integrates **password and checksum management**, ensuring that files are accessible only to authorized users.
    - Provides tips for creating strong, memorable passwords for enhanced security.
6. **Performance Efficiency**
    - Encrypts files quickly with minimal impact on system resources.
    - Optimized for handling multiple files simultaneously, reducing time and effort for users.
7. **Improved Privacy**
    - Ensures that cloud service providers or unauthorized entities cannot access the contents of encrypted files.
    - Protects user data from being exposed to third parties, even during legal or governmental demands.
8. **Resilience Against Attacks**
    - Encrypted files remain secure even if cloud credentials are stolen or browsers are compromised.
    - Ensures that files uploaded to the cloud are unreadable without the correct decryption key.

CHAPTER 4

SYSTEM REQUIREMENTS

1.1 HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or higher
- RAM: 4 GB or higher
- Hard Disk: 500 GB
- Ethernet or wifi connectivity

1.2 SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11, Mac os, Linux
- Browser : Chrome or firefox based browser
- Cloud service : google drive, onedrive
- Compression and Packaging Tools : winrar, 7zip

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 MODULE LIST

1. **Setup and Initialization**
2. **Developing the Core Functionality**
3. **User Interface (UI) Development**
4. **Integrating with Cloud Services**
5. **Testing and Debugging**

5.2 MODULE DESCRIPTION

The implementation of the **Browser Extension for Securing Cloud Storage Files** involves several phases, each ensuring the system functions as intended. Below is the detailed step-by-step breakdown:

1. Setup and Initialization

- **Environment Setup**:
  - Install development tools: Visual Studio Code, Node.js, and Git.
  - Configure the browser's developer mode to test extensions (e.g., enable developer mode in Chrome).

- **Project Initialization**:
  - Create a project directory with subfolders for manifest.json, scripts, styles, and UI components.
  - Initialize a Git repository for version control.

2. Developing the Core Functionality

- **Encryption and Decryption Module**:

    - Implement AES-256 encryption using the Crypto.js library in a JavaScript file (encryption.js).
    - Add password and checksum verification logic for secure file management.
- **File Handling**:
    - Use browser APIs to enable file selection, drag-and-drop functionality, and batch processing.
    - Implement logic for uploading files to cloud storage via APIs (e.g., Google Drive and OneDrive).

3. User Interface (UI) Development

- **HTML Structure**:
    - Design a simple interface with options to upload files, set a master password, and manage encrypted files.
- **CSS Styling**:
    - Ensure a clean, user-friendly layout with drag-and-drop areas and responsive design.
- **Event Handling**:
    - Connect UI elements to encryption, decryption, and upload functions using JavaScript event listeners.

4. Integrating with Cloud Services

- **Cloud API Integration**:
    - Configure APIs for supported platforms (e.g., Google Drive API, OneDrive API).
    - Implement authentication (OAuth 2.0) for secure access to cloud accounts.
- **Automated Upload and Decryption**:
    - Automatically upload encrypted files to user-selected cloud folders.
    - Implement automated decryption on file download based on saved password and checksum.

5. Testing and Debugging

- **Unit Testing**:
    - Test individual modules (encryption, file handling, upload/download) for functionality.

- **Integration Testing**:
  - Verify seamless interaction between encryption, cloud APIs, and the user interface.
- **Performance Testing**:
  - Test encryption and upload speeds for files of varying sizes (e.g., 1MB, 20MB).
- **Security Testing**:
  - Simulate attacks (e.g., brute force, interception) to ensure data security.

6. Deployment

- **Extension Packaging**:
  - Create a ZIP package containing all project files (manifest.json, scripts, styles, and assets).
- **Browser Deployment**:
  - Load the extension into Chrome/Firefox for final testing.
  - Publish the extension on browser stores (e.g., Chrome Web Store) after approval.

7. Documentation and User Guide

- **Technical Documentation**:
  - Document all code modules, including their functionality and APIs used.
  - Provide installation and usage instructions.
- **User Manual**:
  - Create a guide explaining how to install, configure, and use the extension effectively.

KEY CONSIDERATIONS :

1. **Error Handling**:
   - Implement clear error messages for incorrect passwords, failed uploads, or unsupported file types.
2. **Scalability**:
   - Design the system to accommodate additional cloud platforms or encryption features in the future.
3. **User Privacy**:
   - Ensure passwords and sensitive data are not stored in plaintext and are handled securely.

CHAPTER 6

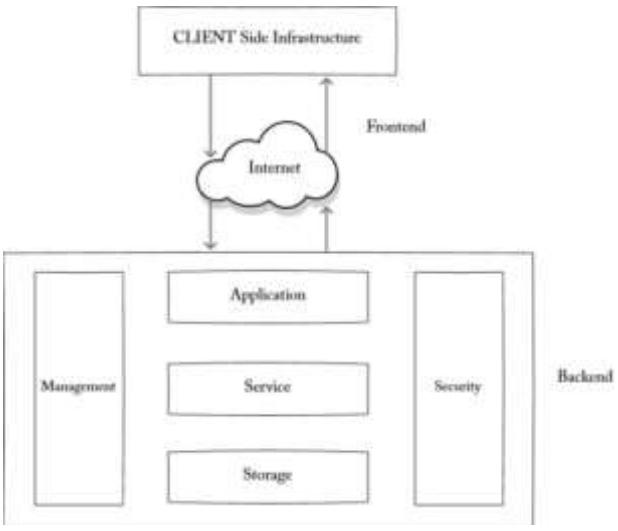
SYSTEM DESIGN

1.1 SYSTEM ARCHITECTURE

The system follows a modular architecture, where each component is designed to perform specific tasks that are integrated to provide an end-to-end secure file management solution. The architecture primarily consists of the **User Interface (UI)**, **Encryption/Decryption Module**, **Cloud Integration Module**, and **Password Management Module**, all communicating with each other to achieve the desired functionality.

At the core of the architecture, the **User Interface (UI)** provides an easy-to-use platform for users to interact with the system. Users can upload files, set a master password, and initiate encryption or decryption actions through simple drag-and-drop actions or manual file selection. The UI is built as a browser extension, allowing it to operate seamlessly across multiple cloud platforms.

Once a file is selected, the **Encryption/Decryption Module** is triggered. This module encrypts files with **AES-256 encryption** before uploading them to the cloud, ensuring data confidentiality. It handles the encryption process using the password provided by the user and generates a checksum for file integrity verification. Upon downloading, the same password is used for decryption, ensuring that the user can access their files securely, even if their cloud storage is compromised.



The **Cloud Integration Module** facilitates communication with cloud platforms such as Google Drive, OneDrive, or Dropbox. It utilizes the respective APIs to authenticate the user and securely upload or download files. This module automatically handles file uploads post-encryption and decryption upon file download, ensuring seamless and automated operations for the user. The system supports multiple cloud platforms, providing flexibility to the users based on their preferences.

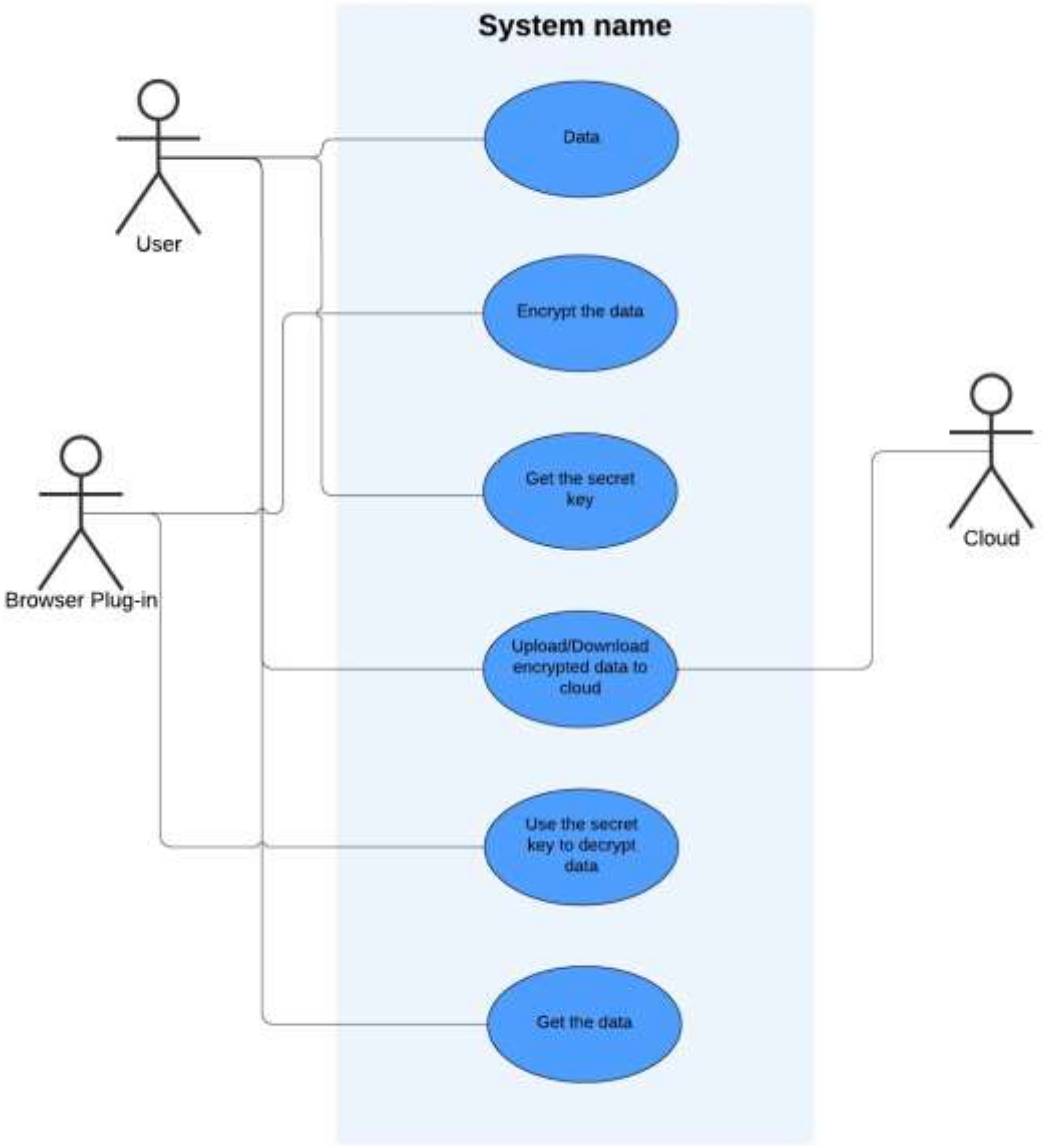
Lastly, the **Password Management Module** ensures that the user's master password and checksum are securely stored and never exposed. This module handles password validation, ensures proper password strength (8-12 characters), and provides tips for creating strong, memorable passwords. It plays a crucial role in the security aspect, ensuring only authorized users can access the encrypted files.

Overall, the architecture ensures a smooth, secure, and user-friendly experience for encrypting, uploading, downloading, and decrypting files in cloud storage, while maintaining a high level of security against potential threats such as unauthorized access or data breaches.

1.2 USE CASE DIAGRAM

A use case diagram is a visual representation that illustrates the interactions between users (actors) and a system, focusing on the system's functional requirements. It highlights the different tasks or functions (use cases) the system performs in response to user interactions.

The **Use Case Diagram** for the Browser Extension for Securing Cloud Storage Files project illustrates the system's key functionalities and the interactions between the primary actors—**the User** and **Cloud Storage** services. The User is the central actor interacting with the system, which provides essential features such as file encryption, decryption, and secure cloud storage management. The cloud storage services (such as Google Drive, OneDrive, or Dropbox) represent the external systems with which the extension integrates to upload and retrieve files.



The user begins by setting a **Master Password** for encryption and decryption purposes. Once set, they can select files to **Encrypt** using the AES-256 encryption algorithm. After encryption, the files are automatically uploaded to the selected cloud storage. The extension communicates with cloud APIs, utilizing OAuth authentication for secure login to the cloud services.

When the user needs to access their files, they can initiate the **Download** process. After downloading the encrypted files from the cloud, the user uses their master password to **Decrypt** them, making the files accessible again. During these processes, the system checks the integrity of the files through **Checksum Verification** to ensure no corruption has occurred during the upload or download.

The extension also provides **Automatic Upload** functionality, which enables files to be uploaded to the cloud once they are encrypted, without additional user input. Throughout the process, the system provides real-time **Status Updates** to the user, indicating the progress of encryption, upload, and decryption activities.

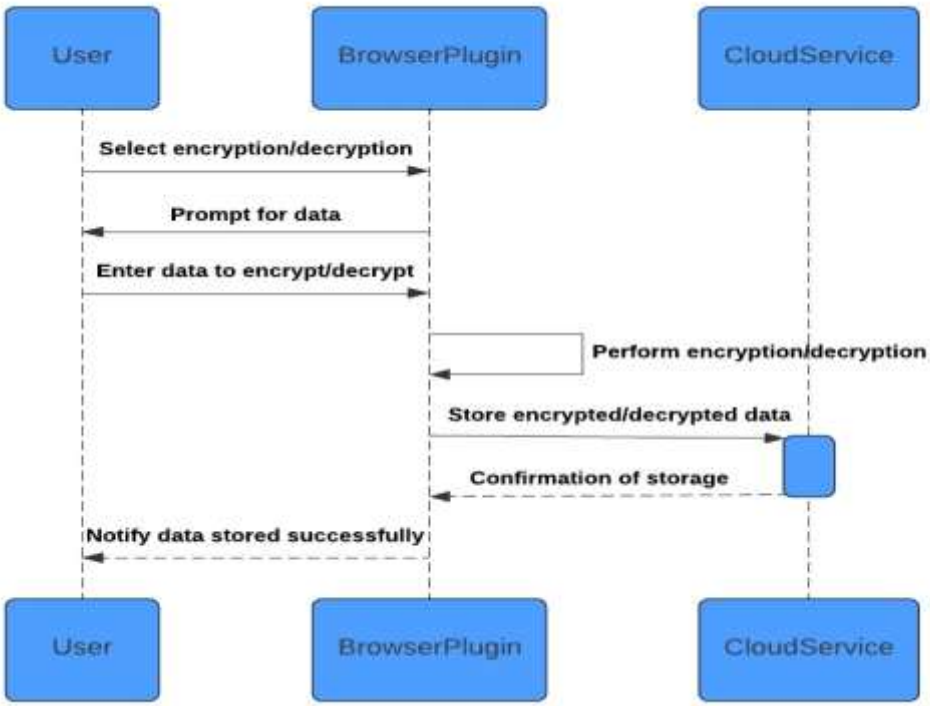
This use case diagram represents the comprehensive flow of interactions within the system, highlighting its user-friendly nature, cloud storage integration, and security-focused design through AES-256 encryption and checksum validation.

1.3 SEQUENCE DIAGRAM :

A sequence diagram visually represents how objects in a system interact with each other over time to complete a specific process or functionality. It focuses on the flow of messages or events between objects, emphasizing the sequence in which these interactions occur.

The diagram consists of objects or actors aligned horizontally, with vertical lifelines representing their timelines and arrows showing messages or method calls exchanged among them.

By illustrating the step-by-step execution of a process, they help developers and stakeholders understand how the system behaves in real-world scenarios.

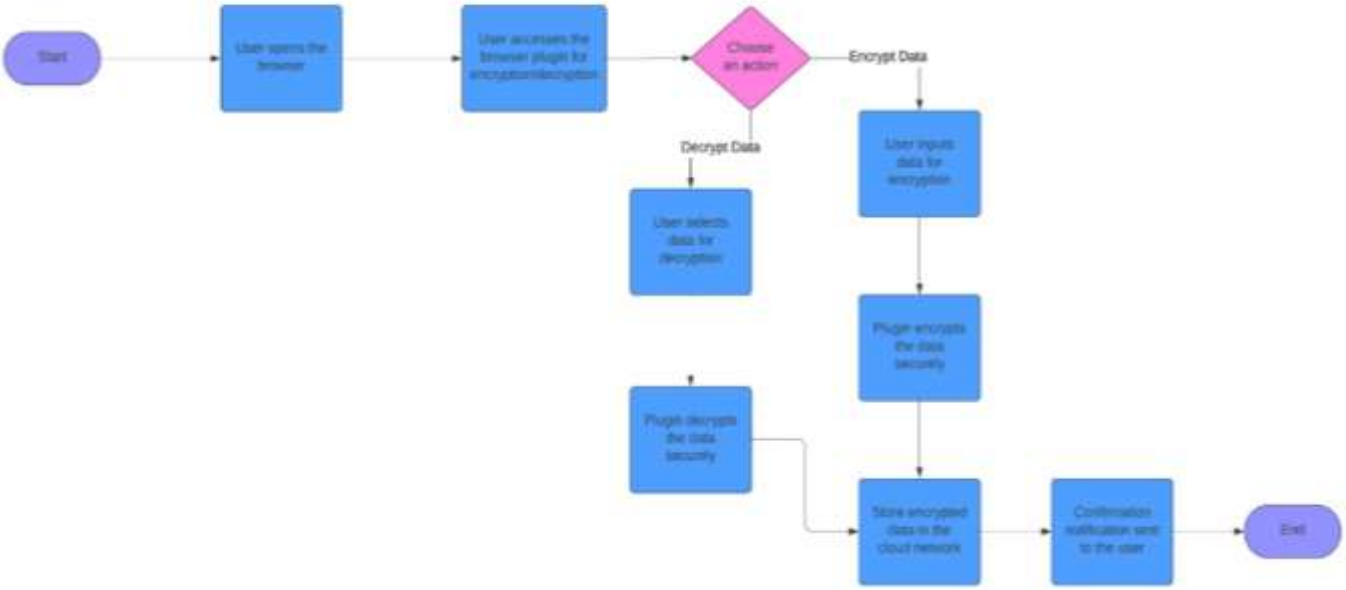


A **Sequence Diagram** visually represents the sequence of interactions between the user and the system components during specific operations. In the context of the Browser Extension for Securing Cloud Storage Files, the diagram shows how the user interacts with the system to perform tasks such as setting a master password, uploading files, and decrypting them.

It outlines the step-by-step process, where the user selects files, initiates encryption, uploads to the cloud, and later decrypts files after downloading. The sequence includes interactions between the user interface, encryption module, cloud storage API, and password management system, illustrating how the system processes these actions in a chronological flow, ensuring secure file handling and communication.

1.4 ACTIVITY DIAGRAM

An activity diagram visually represents the workflow or processes within a system, focusing on the sequence of activities and decision points. It captures the dynamic behavior of the system by illustrating how tasks flow from one to another, often involving parallel processing or branching paths based on conditions.

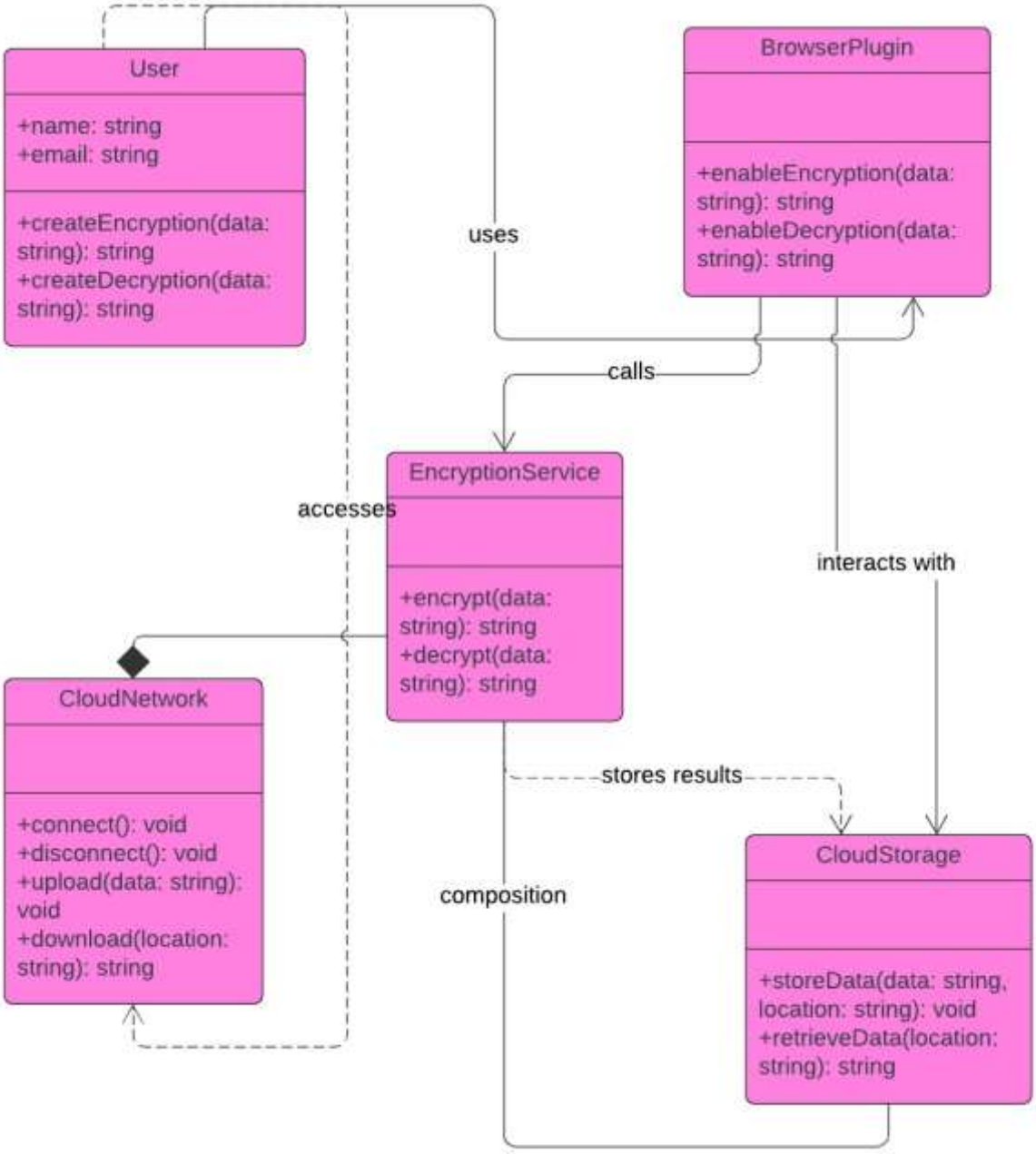


The **Activity Diagram** for the Browser Extension for Securing Cloud Storage Files outlines the flow of activities and operations within the system. It visually represents the sequence of actions that the user performs, starting from selecting files for encryption to uploading them to the cloud, followed by downloading and decrypting files. The diagram also illustrates parallel processes such as checking file integrity and displaying status updates.

Key activities include setting a master password, encrypting files using AES-256, interacting with cloud APIs for file uploads, and ensuring secure decryption. The diagram helps to visualize the system's dynamic behavior, providing clarity on user interactions and system processes.

1.5 CLASS DIAGRAM

A class diagram visually represents the static structure of a system by modeling its classes, their attributes, methods, and the relationships between them. It serves as a blueprint for the system's design, showing how the components of the system are organized and interconnected. Each class is depicted as a rectangle divided into three sections: the top section for the class name, the middle for attributes (data), and the bottom for methods (functions or operations).



Relationships such as associations, generalizations (inheritance), aggregations, and dependencies are shown with lines and specific notations. Class diagrams are fundamental for object-oriented design, helping developers understand the system's structure and how objects interact with one another.

The **Class Diagram** for the Browser Extension for Securing Cloud Storage Files project represents the static structure of the system by showing the key classes, their attributes, methods, and relationships. The main classes in the system include the **User** class, which holds attributes like the master password and methods for setting and validating the password, and the **File** class, which manages file-related operations such as encryption and decryption using AES-256.

Other classes include the **CloudStorage** class, responsible for managing interactions with cloud platforms like Google Drive and OneDrive, and the **Checksum** class, which ensures file integrity during upload and download. These classes are interconnected through associations, such as the File class interacting with CloudStorage for file upload and download processes. The class diagram illustrates the modularity and object-oriented design of the system, ensuring that each functionality is encapsulated within specific classes for better maintainability and scalability.